



SpeakesQuery

TECHNICAL WHITEPAPER

Local-First Data Operations for Analysts and Data Scientists

A single-operator intelligence platform: pipe-based search over columnar storage, sandboxed data ingestion, scheduled analysis, and budget-gated AI - running entirely on hardware you control, with no accounts, no telemetry, and no cloud dependency.

VERSION

1.0.0-rc1

DATE

July 2026

LICENSE

Apache 2.0

AUDIENCE

Data scientists · Data
analysts · Research
engineers

Abstract

SpeakesQuery is a local-first data operations platform that gives a single analyst the working loop of a data team: continuous ingestion from public and authenticated APIs, a pipe-based query language (SPQL) over compressed columnar storage, cron-scheduled searches with e-mail alerting, and large-language-model analysis that is metered, cached, and budget-gated at every call site. Everything - data, credentials, models, logs, and audit history - lives on hardware the operator controls.

This paper describes the design principles behind the platform, its architecture, the feature set as of v1.0.0-rc1, how the pieces compose into end-to-end analytical pipelines, and the hardware required to run it - from a laptop to a LAN with a dedicated GPU inference host.

56

 SPQL
COMMANDS

52

 BUILT-IN
FUNCTIONS

135

 READY-MADE
CONNECTORS

5,700+

 AUTOMATED
TESTS

0

 TELEMETRY
ENDPOINTS

Contents

1	The Problem	3
2	Design Principles	4
3	Architecture	6
4	Platform Capabilities	8
5	The Intelligence Layer: Metered AI	10
6	How It Composes: A Working Day	12
7	Security & Privacy Posture	13
8	Hardware & Deployment	14
9	Getting Started	16

1 · The Problem

Analysts across finance, public health, energy, policy, and research science share a working loop: pull data from a dozen sources on a schedule, filter and aggregate it, watch for anomalies, and write up what matters. The tooling for that loop has drifted into two unsatisfying corners.

The enterprise corner - Splunk-class log platforms and Bloomberg-class terminals - delivers the right interaction model (a pipe language over indexed data, scheduled alerting, operational dashboards) at a per-seat or per-gigabyte price that excludes individuals and small teams, and with an architecture that requires shipping your data to someone else's infrastructure.

The DIY corner - a folder of Jupyter notebooks, cron jobs, and CSV files - keeps the data local but abandons the platform properties that make the loop reliable: scheduling with an audit trail, alerting that distinguishes a quiet day from a broken pipeline, storage that enforces size budgets, credential handling that isn't a plaintext `.env` file, and any notion of cost control once LLM calls enter the workflow.

The gap widened when large language models became part of daily analysis. Cloud AI assistants are powerful but unmetered by default - a scheduled job that silently retries a failing LLM call can burn a month's budget overnight - and every prompt ships potentially sensitive data to a third party. Self-hosted models eliminate both concerns but arrive with no orchestration: no scheduling, no journaling of what was asked and answered, no cost accounting to compare against cloud alternatives.

SpeakesQuery's thesis: a single operator on a single machine can run a multi-source intelligence operation that previously required a team - if the platform makes scheduled ingestion, pipe-based analysis, semantic search, and cost-gated AI economical to express in one query language, and if all of it runs where the data already lives.

2 · Design Principles

Eight principles govern every subsystem. They are enforced by convention documents, schema-drift tests, and - where money or data loss is at stake - by dedicated regression tests that fail the build when a principle is violated.

1 Local-first, zero telemetry

The engine, the UI, the scheduler, the vector index, and the audit stores all run on the operator's hardware. The software contains no analytics, no phone-home, no account system. The only network traffic is the traffic you configure: your data sources, your SMTP server, and (optionally) your chosen LLM endpoints.

2 Open formats, no lock-in

Data lands in gzip-compressed Apache Parquet and SQLite - readable by pandas, DuckDB, Polars, R, or anything else, forever. Configuration is YAML. If you stop using SpeakesQuery tomorrow, every byte of your data remains fully usable.

3 A familiar language, not a new one

SPQL is a pipe-delimited search language in the tradition analysts already know from Splunk: `index="..." | search ... | stats count by ... | sort ... | head 10`. The 56 commands and 52 functions are defined in a formal ANTLR4 grammar; the grammar is the single source of truth that also powers console autocomplete and the visual query builder.

4 Untrusted code runs sandboxed

Ingestion scripts execute inside RestrictedPython with a curated module allowlist, an outbound-domain allowlist, per-run resource budgets (rows, bytes, time) that scale inversely with schedule frequency, and no filesystem or subprocess access. A separate, explicitly opt-in "pro" tier grants the full scientific stack (scipy, scikit-learn, numpy, duckdb) for scripts the operator chooses to trust.

5 Every billable call is gated, metered, and journaled

All LLM traffic flows through single choke points that enforce budgets (`max_cost_usd`), support zero-cost previews (`dry_run`), record full request/response history, and cache by content hash so idempotent re-runs are free. The test suite includes "money-leak canaries": tests that patch the billable client to explode, exercise every disabled/dry-run path, and assert zero invocations.

6 Long-horizon data is append-only and additive-only

Records meant to be consulted years later - trade journals, decision logs, telemetry - live in a protected `IMMUTABLE/` namespace that the storage janitor never touches. Schemas there may gain columns but never lose them, enforced by frozen-snapshot tests, so a query written today still runs against a decade of history.

7 Observability is just more data

Search runs, ingestion runs, alert dispatches, configuration changes, and every AI call (with token counts and dollar cost) are written as Parquet log streams - queryable with the same SPQL as any dataset. "How much did model X cost this week, by alert group?" is a one-line query, not a support ticket.

8 Crash-safe by construction

Every persistent write - Parquet, YAML, settings, indexes - stages to a temporary sibling and atomically renames into place. A power cut mid-write leaves the previous consistent state, never a torn file. Deleted configurations are soft-deleted with a 30-day recovery window.

3 · Architecture

SpeakesQuery is a single Python 3.12 application (deployed as one Docker container) organized as four cooperating planes over a shared storage layer.

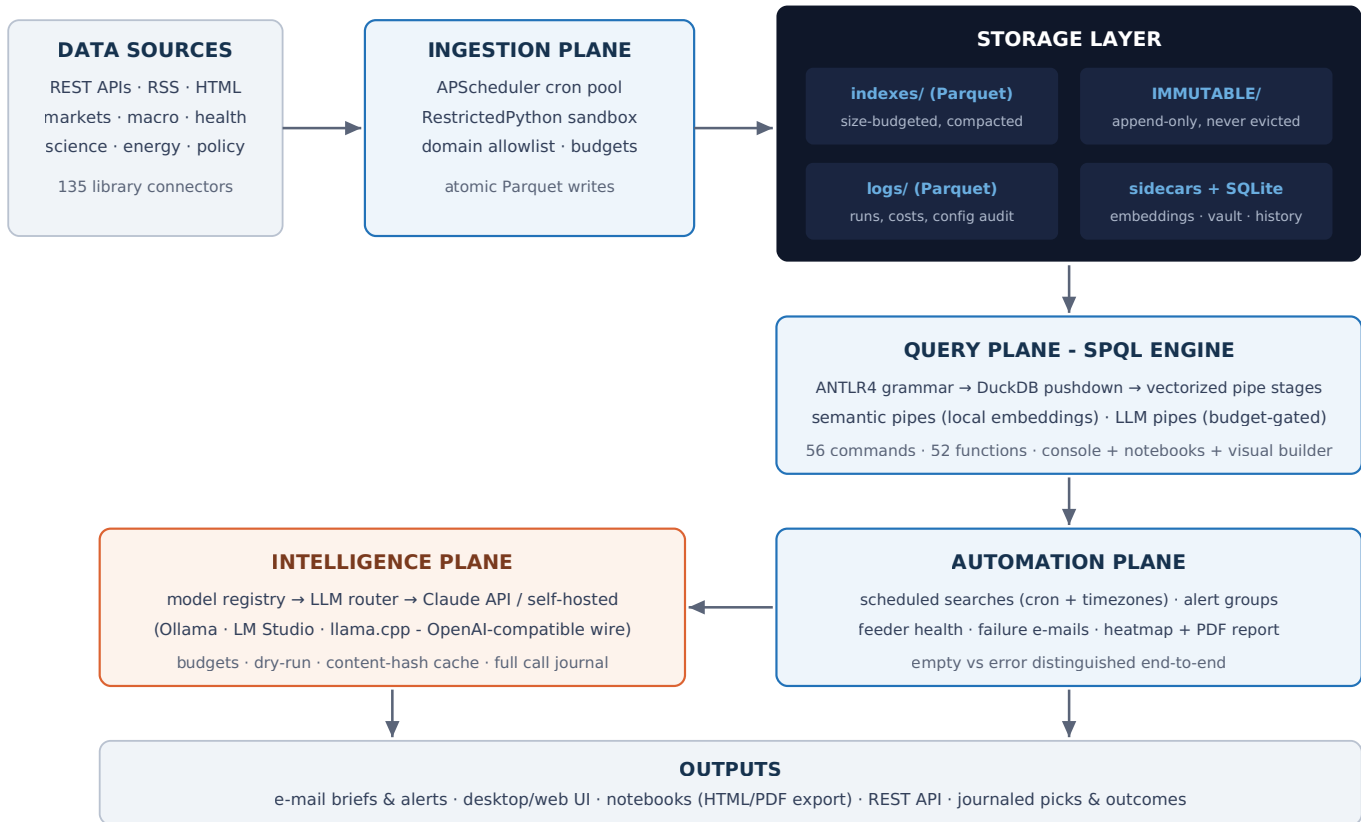


Figure 1 - SpeakesQuery's four planes over the shared storage layer. Every arrow is local; the only outbound traffic is source APIs, SMTP, and the LLM endpoints you configure.

3.1 Storage

Ingested data lands as gzip-compressed Parquet under `indexes/`, one subdirectory per source, with a mandatory Unix-epoch column that powers time-bounded queries. Total and per-subdirectory disk budgets are enforced by a compaction and cleanup job - with two carve-outs: the `logs/` stream has its own independent budget (noisy logging can never evict data), and `IMMUTABLE/` is never cleaned at all. Reference data (SQLite, CSV, JSON) is queryable alongside Parquet through the same language.

3.2 Query execution

SPQL parses through a generated ANTLR4 parser. The initial `index=` clause compiles to DuckDB reads with predicate and projection pushdown - so `earliest= / latest=` bounds and field filters skip Parquet row groups rather than scanning them. Subsequent pipe stages execute as vectorized pandas

operations. Every pipe handler treats an empty frame as a valid state, so a quiet day propagates cleanly instead of raising.

3.3 Scheduling and dispatch

Two APScheduler-based engines drive automation: one executes ingestion scripts in the sandbox, the other runs saved searches and alert groups on cron schedules with per-job IANA timezones (a schedule authored as `30 10,15 * * mon-fri` in `America/New_York` survives DST transitions untouched). Alert groups aggregate up to ten searches into a single LLM dispatch and deliver the result as an e-mail brief, with per-group budgets, failure alerting on a separate administrative channel, and a per-feeder health resolver that can diagnose a broken pipeline layer by layer.

4 · Platform Capabilities

4.1 SPQL - the query language

SPQL covers the working vocabulary of investigative analytics: filtering (`search` , `where` , `regex`), derivation (`eval` with 52 functions, `rex` field extraction), aggregation (`stats` , `eventstats` , `streamstats` , `timechart` , `bin`), reshaping (`table` , `fields` , `mvexpand` and a full multivalue toolkit), joining (`join` , `append` , `lookup` , `multisearch`), and result management (`dedup` , `sort` , `head` , `outputlookup` , `loadjob`).

```
# Which agencies' contract awards spiked this week vs. their 90-day baseline?
index="indexes/gov/contract_awards/*.parquet" earliest="-90d"
| eval week = strftime(_epoch, "%Y-%W")
| stats sum(award_usd) as weekly_usd by agency, week
| eventstats avg(weekly_usd) as baseline by agency
| where weekly_usd > 2 * baseline
| sort -weekly_usd | head 20
```

4.2 Ingestion and the connector library

Scripts are authored, linted (an in-browser Pyflakes pass with no server round-trip), tested against the real sandbox, and scheduled from the UI. The shipped library provides 135 tested connectors - 101 of which need no API key - spanning prediction markets (Polymarket, Kalshi, Metaculus, Manifold), macroeconomics (FRED, Treasury, World Bank), equities and options (SEC EDGAR, earnings calendars, IV/skew/term-structure suites), crypto (CoinGecko, DeFi Llama), public health (FDA, ClinicalTrials.gov), energy (EIA), climate and hazards (NOAA, USGS), geopolitics (GDELT), science funding (NIH, arXiv), sentiment (Reddit, Hacker News), and more. Every connector carries schema-validated tests with mocked responses, and the platform's schema-drift conventions catch upstream payload changes.

4.3 Semantic search

A local sentence-transformers model (all-MiniLM-L6-v2, 384 dimensions, ~80 MB) powers two pipes: `| nearest "hawkish central bank commentary"` ranks rows by meaning rather than keywords, and `| dedup_semantic` collapses near-duplicate stories across sources. A background sweeper pre-computes embedding sidecar files per index, so semantic queries read precomputed vectors instead of embedding at query time. No embedding API, no per-token cost, no data leaving the machine.

4.4 Scheduled searches and alert groups

Any query can be promoted to a scheduled search with a cron expression, lookback window, and e-mail alert. Alert groups compose multiple searches into one LLM-analyzed brief on its own schedule - the platform's flagship pattern (Section 6). Reusable boilerplate prompts, per-group recipient lists (including `@group` mailing lists), row caps, and token estimation keep dispatches predictable.

4.5 Notebooks and the visual builder

Notebook mode chains typed cells - SPQL, LLM, markdown - where each cell's output feeds the next, with content-hash reactive caching so editing one prompt re-runs only what changed. A

`promote_to_alert_group` cell shows a dry-run preview in-notebook and deploys to production only through an explicit endpoint - a config-leak canary test guarantees notebook execution can never silently mutate production. The visual builder offers the same grammar as a drag-drop canvas with lossless round-trip to text, as the on-ramp for colleagues without SPQL fluency.

4.6 Operations: logs, schedule intelligence, backup

Every run of everything is a Parquet row you can query. The Schedule page renders firing-count and data-volume heatmaps (day-of-week × hour), recent-activity charts, and exports a branded PDF operations report with per-alert-group feeder health and anomaly buckets - failing runs, never-ran jobs, empty output, latency outliers. A stdlib-only persistence tool snapshots, backs up, restores, and diffs all user data; the update script takes a pre-update snapshot and verifies a post-update diff automatically.

5 · The Intelligence Layer: Metered AI

SpeakesQuery treats LLM calls the way a database treats disk I/O: a metered resource behind a uniform interface, never an unaccounted side effect.

5.1 The model registry and router

Models are YAML records - provider, endpoint, per-token costs, output ceilings, timeouts, and sampling parameters. A provider-agnostic router dispatches every call by model id: the Anthropic Claude API for frontier-quality analysis, or any self-hosted server speaking the industry-standard Chat Completions wire format - LM Studio, llama.cpp, vLLM - plus Ollama's native API. Swapping an alert group's analysis from a cloud model to a LAN GPU host is a one-field change, and cost accounting stays comparable because both paths land in the same journal with the same schema.

5.2 Composable LLM pipes

Because `| llm` is a pipe stage rather than a terminal step, model calls compose with everything else:

```
# Classify only what survives the cheap filters; never spend more than 50¢
index="indexes/news/articles/*.parquet" earliest="-1d"
| dedup_semantic title
| where relevance_score > 0.7
| llm model="local-8b" prompt="One-line risk assessment: $title$ - $summary$"
  max_cost_usd=0.50 dry_run=false
```

Four higher-order pipes orchestrate multi-model logic: `llm_route` (confidence-based cost cascades - a cheap local model answers first, escalating to an expensive model only on low confidence), `llm_refine` (drafter/critic loops), `llm_ensemble` (multi-model voting), and `llm_until` (convergence loops with hard cost ceilings). Every billable pipe honors `max_cost_usd` and `dry_run` - the dry run prices the exact dispatch without making it.

5.3 Cost control as an architectural property

Layer	Mechanism
Before dispatch	Cost estimation + <code>max_cost_usd</code> budget gates; <code>dry_run</code> previews; per-alert-group dollar budgets; daily analyzer budget
At dispatch	Single router choke point; retry policy that excludes timeouts (a retry against the same ceiling only doubles spend); optional prompt-compression proxy with fail-open semantics
After dispatch	Content-hash cache (identical re-runs are free and served locally); full request/response journal in SQLite; per-call cost rows in the Parquet log stream, SPQL-queryable
In the test suite	Money-leak canaries: billable clients are patched to raise on invocation, every disabled/dry-run path is exercised, zero invocations asserted

Practical effect: a daily multi-source brief that costs a few cents on a frontier cloud model - or \$0 marginal on a LAN inference host - with a queryable ledger proving which. Runaway spend is a failing test, not a billing surprise.

6 · How It Composes: A Working Day

The subsystems exist to serve one loop. Here is a representative production configuration - an options-market analyst's day - drawn from the platform's own dogfooding.

When (ET)	What runs	Which subsystem
Overnight	Twenty-plus ingestion scripts pull markets, filings, macro series, and news on staggered crons; embedding sweeper refreshes sidecars	Ingestion plane · sandbox · semantic layer
~10:00	Six signal feeders run as saved searches - IV rank extremes, term-structure inversions, skew, pre-earnings implied moves, unusual activity - each a pure SPQL query over the night's data	Query plane · scheduler
10:30	The alert group serializes all six result sets into one prompt, dispatches to the configured model, parses the structured picks from the response, e-mails the brief	Alert groups · LLM router
10:30 + ε	Every pick is journaled - instrument, direction, entry, stops, thesis, and the exact feeders that sourced it - to the append-only IMMUTABLE store	IMMUTABLE namespace
Daily	A deterministic tracker marks every open pick to market and grades closures against the rules as they existed at entry - no hindsight	Ingestion plane (pro tier)
Sunday	A weekly review aggregates outcomes into hit rate and calibration; the operations PDF reports every job's health, cost, and anomalies	Alert groups · schedule intelligence

Nothing in this loop is bespoke code. It is configuration over the same primitives available for any domain: swap the feeders and the prompt, and the identical machinery runs epidemiological surveillance over FDA and ClinicalTrials feeds, grid-stress monitoring over EIA and NOAA data, or policy tracking over the Federal Register and GDELT. The platform's own extension record demonstrates the claim - a complete personal media-curation pipeline (multi-source candidate ingestion, an LLM playlist composer, viewing-telemetry feedback) shipped as pure configuration plus a thin REST contract, with zero engine forks.

The compounding property: every layer feeds the ones above it. Ingestion feeds search; search feeds schedules; schedules feed alert groups; alert groups journal to IMMUTABLE; IMMUTABLE feeds the weekly review that tunes the prompts; and the logs of all of it are themselves data for the next investigation.

7 · Security & Privacy Posture

Surface	Control
Data residency	All data, indexes, embeddings, and audit history remain on operator hardware. No accounts, no telemetry, no third-party analytics. Outbound traffic is limited to configured sources, SMTP, and chosen LLM endpoints.
Credentials	Fernet-encrypted vault (SQLite); master key stored outside the repository with <code>0600</code> permissions; decryption scoped to a single script run; key-rotation tool re-encrypts the vault in place. Vault mutations are audit-logged, and audit-log failures are never silently swallowed.
Untrusted code	RestrictedPython sandbox: module allowlist, no filesystem/subprocess access, no dunder access, resource budgets per run. Outbound HTTP restricted by a regex domain allowlist enforced at the engine layer - pro-tier scripts included.
Secrets in transit to LLMs	Secret-shaped tokens (API-key patterns) are regex-scrubbed from every request/response body before journaling, guarding against accidental key paste into prompts.
Web UI	Server binds LAN-configurable (localhost-only via one env var); all rendering escapes HTML; error responses never expose tracebacks.
Supply chain	Single container image built locally from source; Python dependencies pinned; static analysis (bandit, flake8) in the development gate alongside the 5,700-test suite.

Scope note: v1.0.0-rc1 targets single-operator deployments on trusted networks. Multi-user authentication, TLS termination, and role separation are on the public roadmap (Phase 6) and out of scope for this release.

8 · Hardware & Deployment

SpeakesQuery's core is deliberately light: DuckDB and pandas over compressed Parquet on one machine. The only meaningfully heavy option - local LLM inference - is optional and lives on whatever second machine you point the model registry at.

8.1 Requirements

	Minimum	Recommended
CPU	2 cores, x86-64 or ARM64	8 cores (parallel ingestion + DuckDB scans + embedding batches)
Memory	8 GB	16-32 GB (comfortable headroom for wide multi-file scans and the embedding model)
Storage	20 GB free	100+ GB NVMe SSD (gzip-Parquet is compact - budget ~1-5 GB/year per busy source - but SSD latency dominates query feel)
OS	Any Docker host - Linux, macOS (Intel or Apple Silicon), Windows via WSL2. Raspberry Pi 5-class ARM boards run the core acceptably for light schedules.	
Network	Outbound HTTPS to your data sources and SMTP. No inbound ports required beyond the UI (default 5111, LAN-configurable).	

The semantic layer adds roughly 1 GB of working memory when active (the MiniLM model plus batch buffers). Disk budgets are enforced by the platform itself - you set `max_indexes_size_gb` and the janitor holds the line.

8.2 Intelligence-tier options

Tier	Hardware	What it gets you
Cloud API only	Nothing extra	Frontier-model quality (Anthropic Claude) for briefs and analysis; typical daily-brief workloads run cents per day, fully metered
Small local	16 GB unified memory or a 12 GB GPU on any LAN box	4-8B-class models via Ollama/LM Studio for classification, tagging, and routing tiers of cost cascades - \$0 marginal
Large local	Dedicated inference host: 48-128 GB unified memory (Apple Silicon) or equivalent GPU VRAM	30-120B-class models handle full daily-brief analysis at \$0 marginal cost; SpeakesQuery drives them over the LAN exactly like the cloud path, same journal, same budgets

Mixed fleets are the intended end state: route high-volume, low-stakes calls to local tiers and reserve cloud calls for the judgments that warrant them - expressible per-alert-group or per-pipe (`llm_route`).

8.3 Deployment shape

One Docker container, one command (`./install.sh` - preflight checks, `.env` generation, build, launch). All user data bind-mounts to the host tree, so container rebuilds never touch data. An `update.sh` wrapper performs pull → snapshot → rebuild → verify, taking a pre-update backup and diffing user data after the update as a regression gate. Non-Docker native installs are supported for development via `setup.sh` .

9 · Getting Started

```
# 1. Clone and install (Docker required - that's the only prerequisite)
git clone https://github.com/13alvone/speakesquery.git
cd speakesquery
./install.sh
# 2. Open http://localhost:5111 - deploy a no-key connector from the
#   Script Library (101 of 135 need no credentials), watch data land,
#   and run your first SPQL query against it.
```

A practical first hour: deploy two or three library connectors in your domain, let them fire once, explore the indexes from the console with autocomplete, promote a useful query to a scheduled search with an e-mail alert, then wrap several searches into an alert group with a boilerplate prompt. The in-app Help serves the complete documentation set - language fundamentals, cookbook, application guide, and per-subsystem references.

Project posture

SpeakesQuery is authored infrastructure released under the Apache License 2.0. It is intentionally non-rent-seeking: the core engine is free by design and remains so. There is no hosted version, because a hosted version would contradict the premise. The project survives on correctness, clarity, and trust - properties you can verify, since every line that touches your data is inspectable on your own disk.

SpeakesQuery Technical Whitepaper · v1.0.0-rc1 · July 2026 · Apache License 2.0 · Copyright 2025–2026 Chris (13alvone) Speakes. Product names mentioned (Splunk, Bloomberg, Anthropic Claude, Ollama, LM Studio) are trademarks of their respective owners; references are descriptive only.